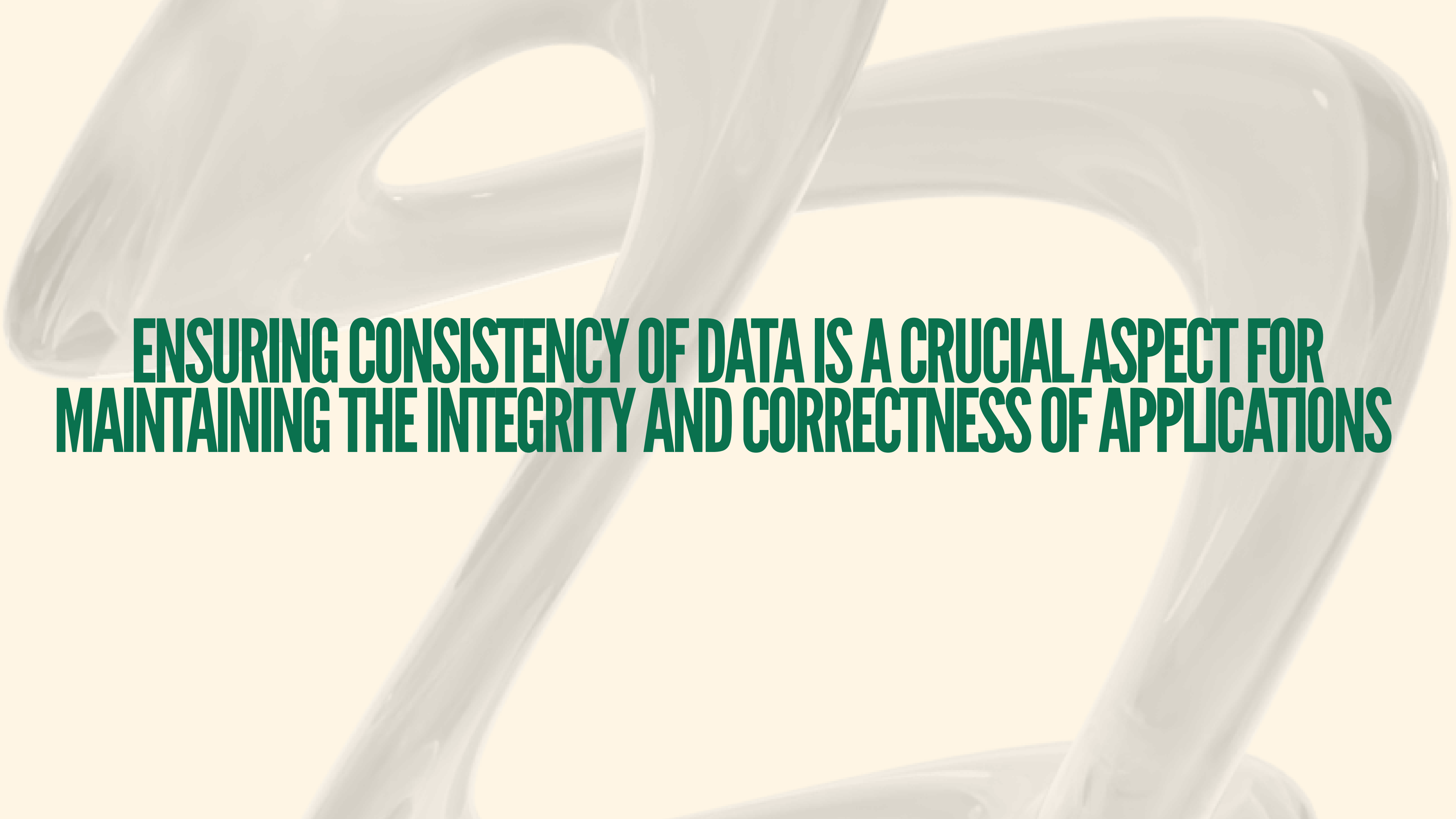


CONSISTENCY ALGORITHMS





**ENSURING CONSISTENCY OF DATA IS A CRUCIAL ASPECT FOR
MAINTAINING THE INTEGRITY AND CORRECTNESS OF APPLICATIONS**

CONSISTENCY ALGORITHMS

Consistency algorithms designed to help maintain consistency in distributed systems by providing a way to generate timestamps for events that occur across multiple nodes.

These algorithms define rules for how data should be replicated across multiple nodes, how updates should be propagated, and how conflicts should be resolved.

The most common clock-based consistency algorithms

- Lamport Clock
- Vector Clocks
- Scalar Clocks
- Berkeley algorithm

LAMPORT CLOCKS



Lamport Clocks are a type of logical clock that use a single counter to assign a timestamp to each event, which reflects the order of events in a distributed system.

Lamport Clocks can be used to establish the ordering of events that occur on different nodes in the system, but they cannot capture the causal relationship between events.

We say that something happened at 3:15 if it occurred after our clock read 3:15 and before it read 3:16 $[3:15, 3:16[$



ALGORITHM STEPS:

A Lamport clock is a numerical software counter maintained by each independent process.

The algorithm for updating the counter follows some simple rules:

- A process increments its own counter before each event (e.g., before sending an update message);
- When a process sends a message, it includes its counter value with the message;
- On receiving a message, the recipients counter is updated to the larger of its own counter or the one received by the incoming message

PSEUDOCODE

The algorithm for sending can be expressed as:

```
# event happens  
time = time + 1;  
send(message, time);
```

The algorithm for receiving a message as:

```
(message, incoming_time) = receive();  
time = max(incoming_time, time) + 1;
```

EXAMPLE

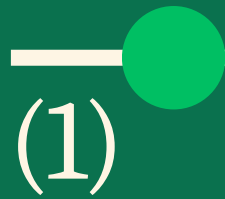
P3 (0) —————

P2 (0) —————

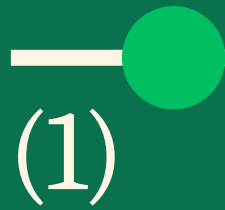
P1 (0) —————

EXAMPLE

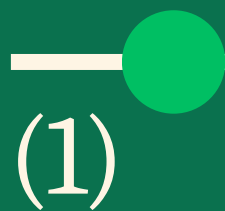
P3



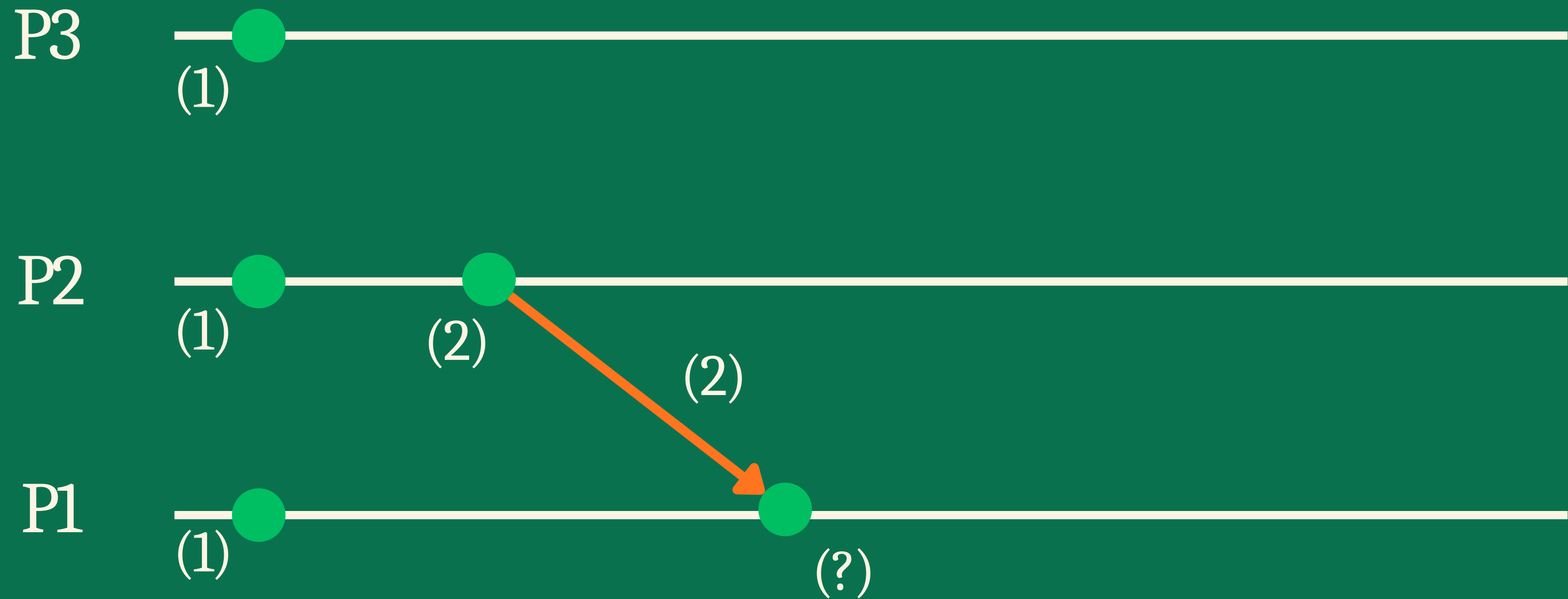
P2



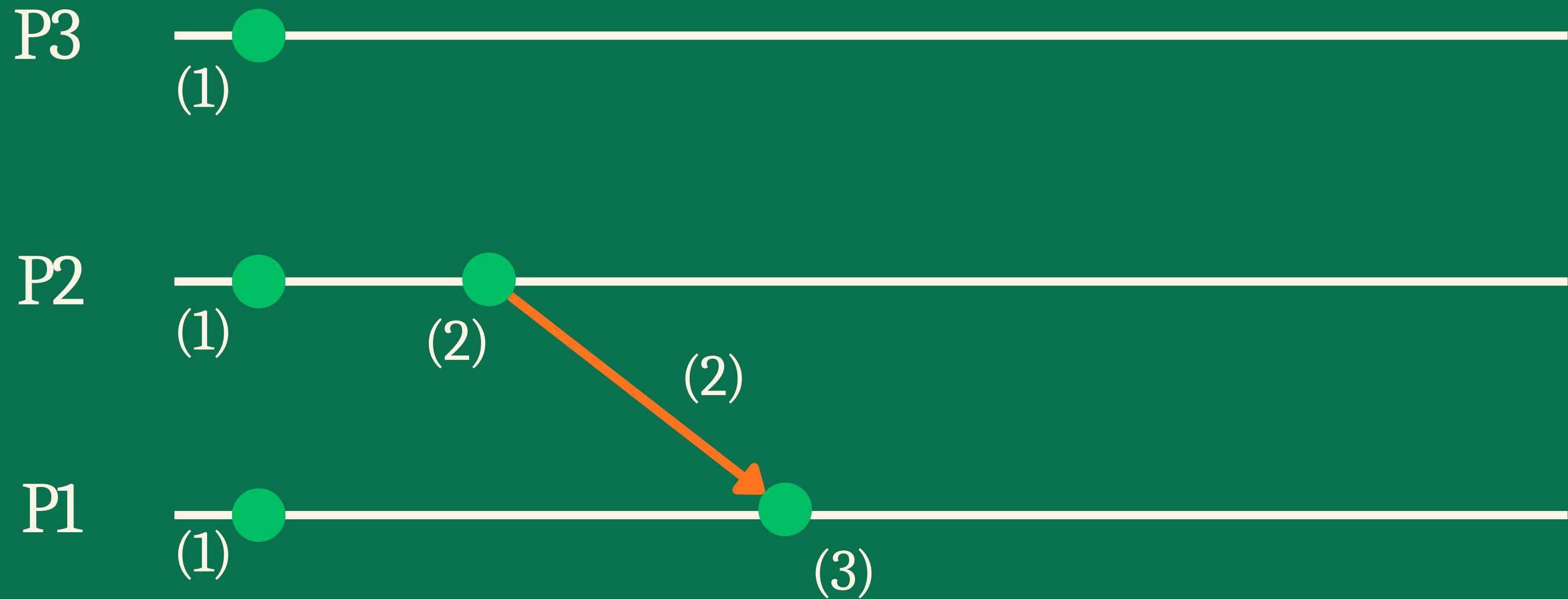
P1



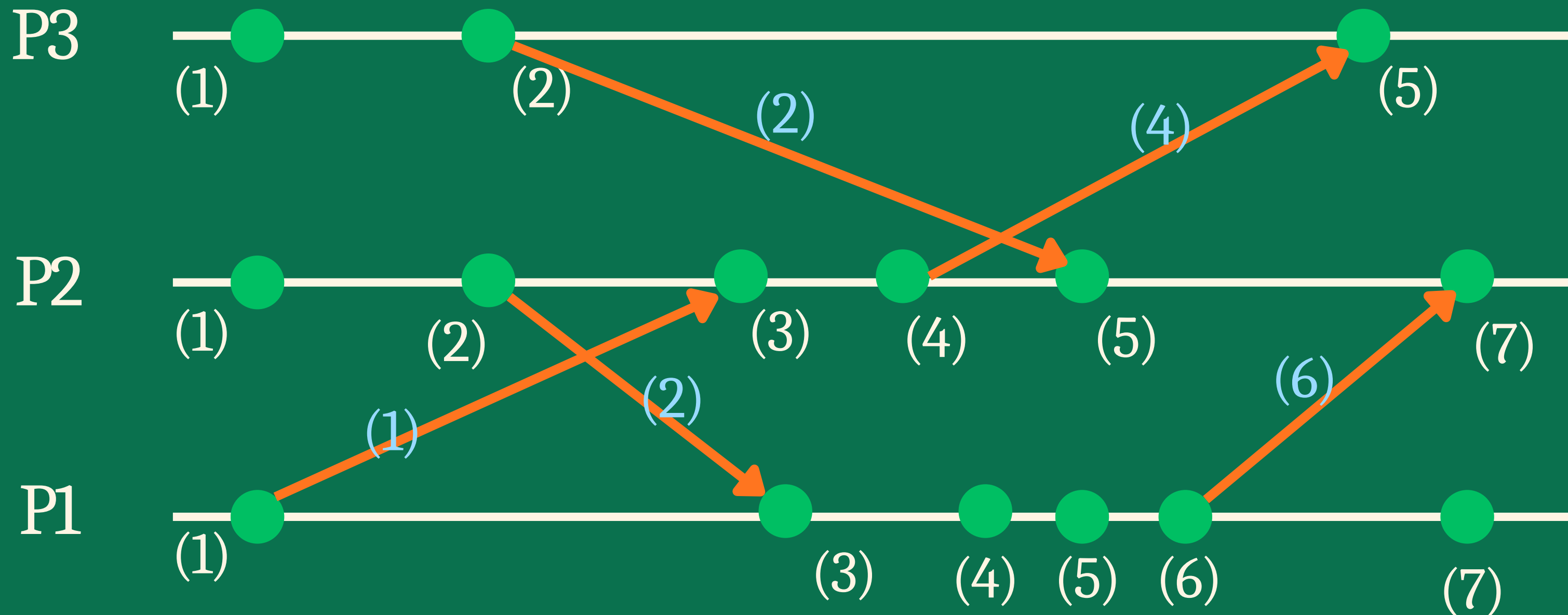
EXAMPLE



EXAMPLE



EXAMPLE



VECTOR CLOCKS



Vector Clocks are a type of logical clock that extends the concept of Lamport Clocks by using a vector to assign a timestamp to each event.

Vector Clock is an algorithm that generates partial ordering of events and detects causality violations in a distributed system.

Instead of assigning a single timestamp to each event, Vector Clocks assigns a vector of timestamps, with one entry for each process in the system.



ALGORITHM STEPS:

Initially:

all the clocks are set to zero.

Every time an Internal event occurs in a process:

the value of the processes's logical clock in the vector is incremented by 1

Every time a process sends a message:

the value of the processes's logical clock in the vector is incremented by 1.

EXAMPLE

P1 [0,0,0]



P2 [0,0,0]



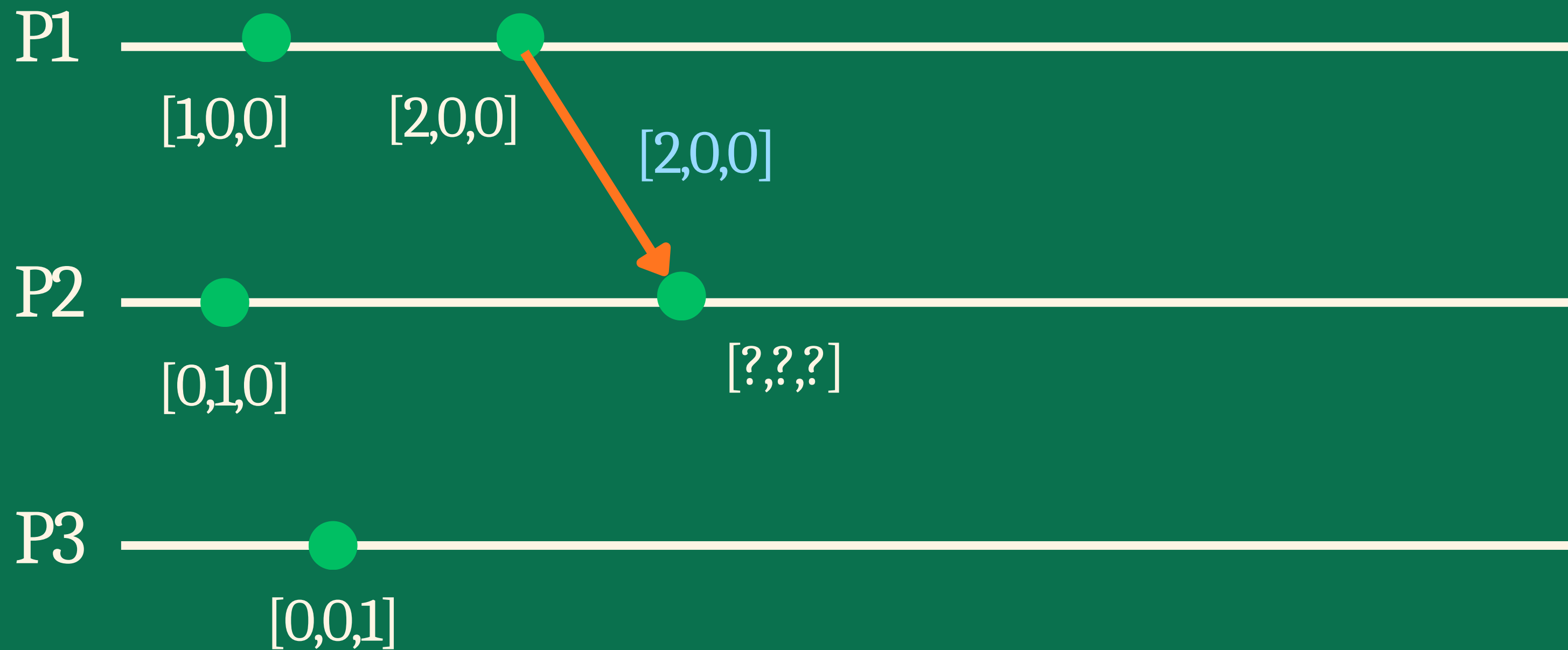
P3 [0,0,0]



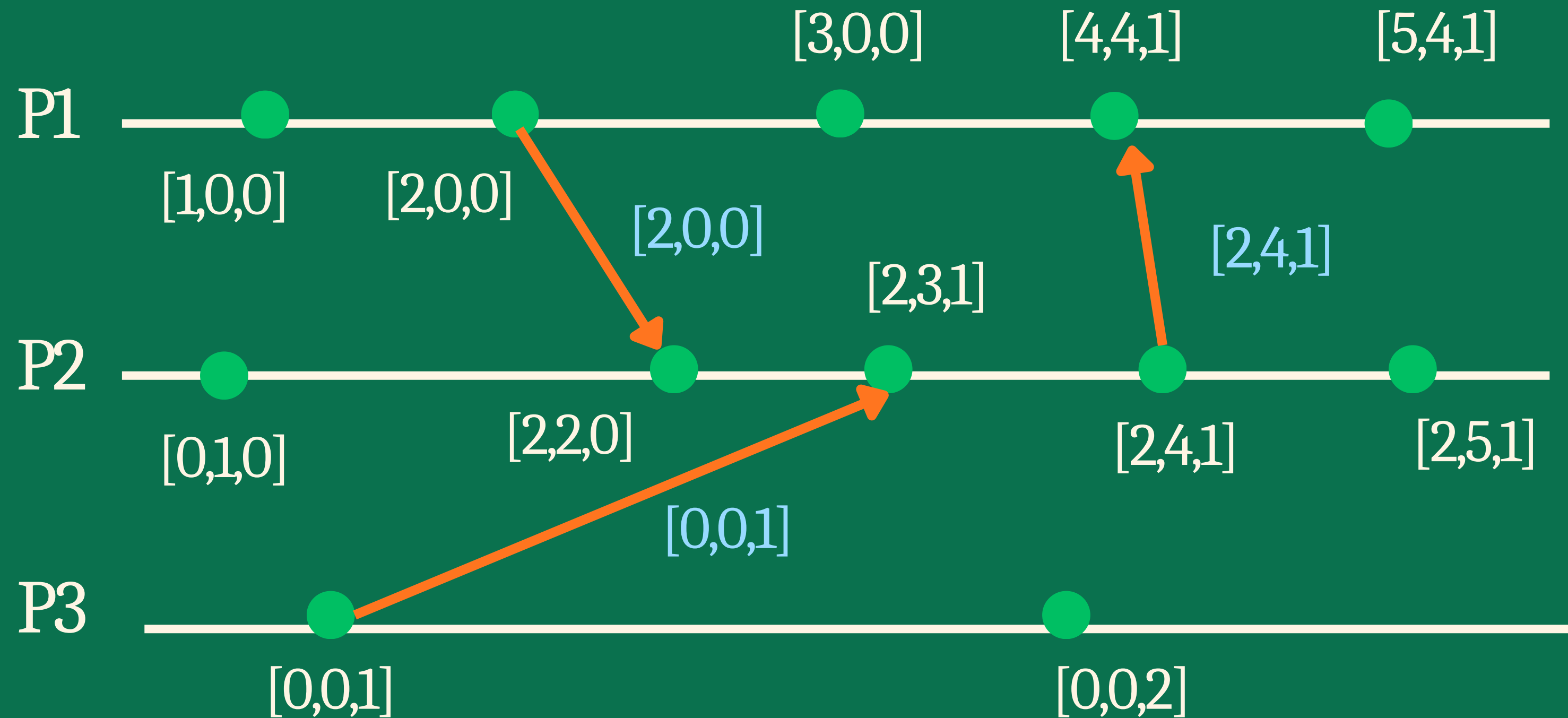
EXAMPLE



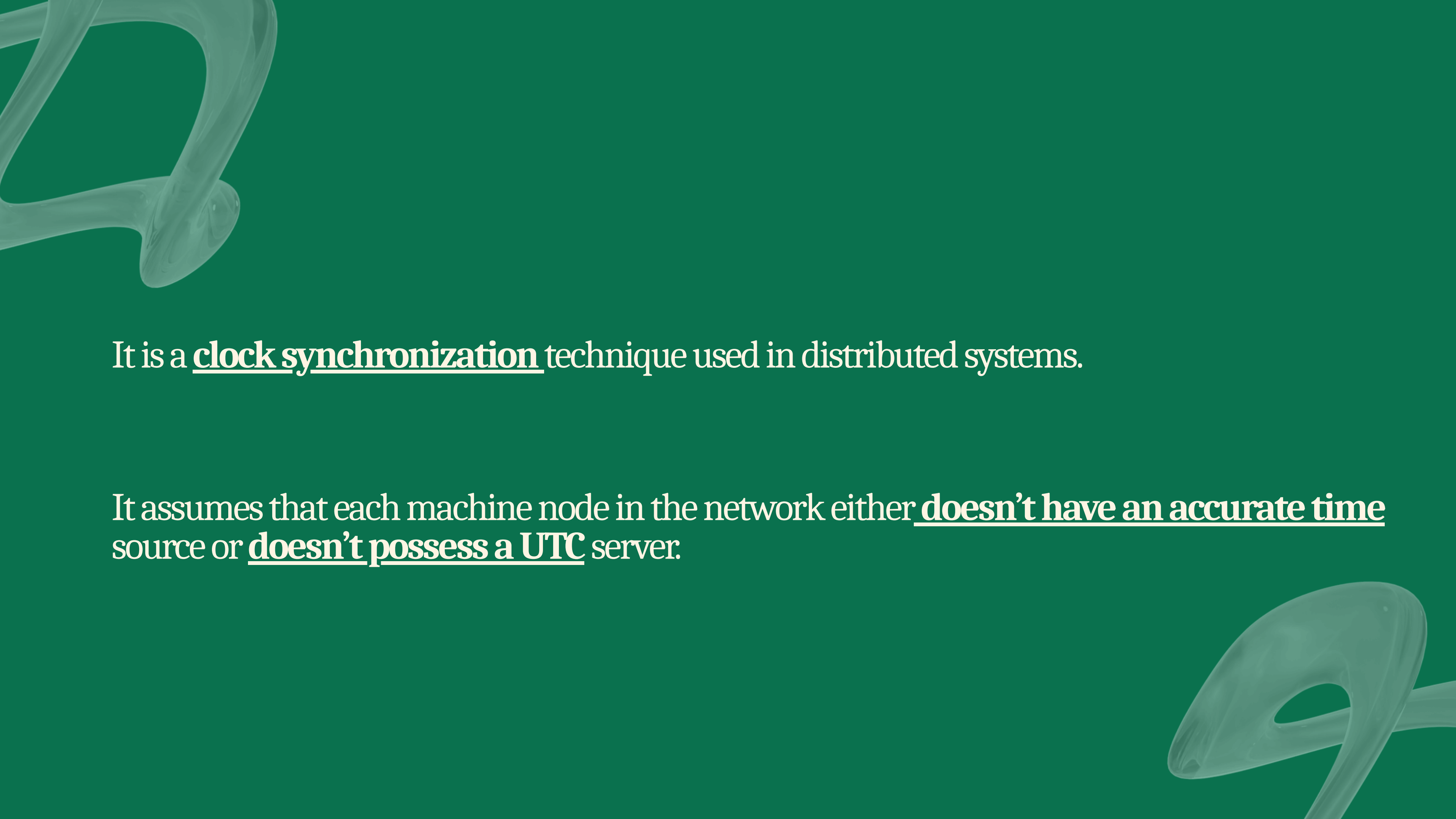
EXAMPLE



EXAMPLE



BERKELEY'S ALGORITHM



It is a clock synchronization technique used in distributed systems.

It assumes that each machine node in the network either doesn't have an accurate time source or doesn't possess a UTC server.

ALGORITHM STEPS:

An individual node is chosen as the master node from a pool node in the network

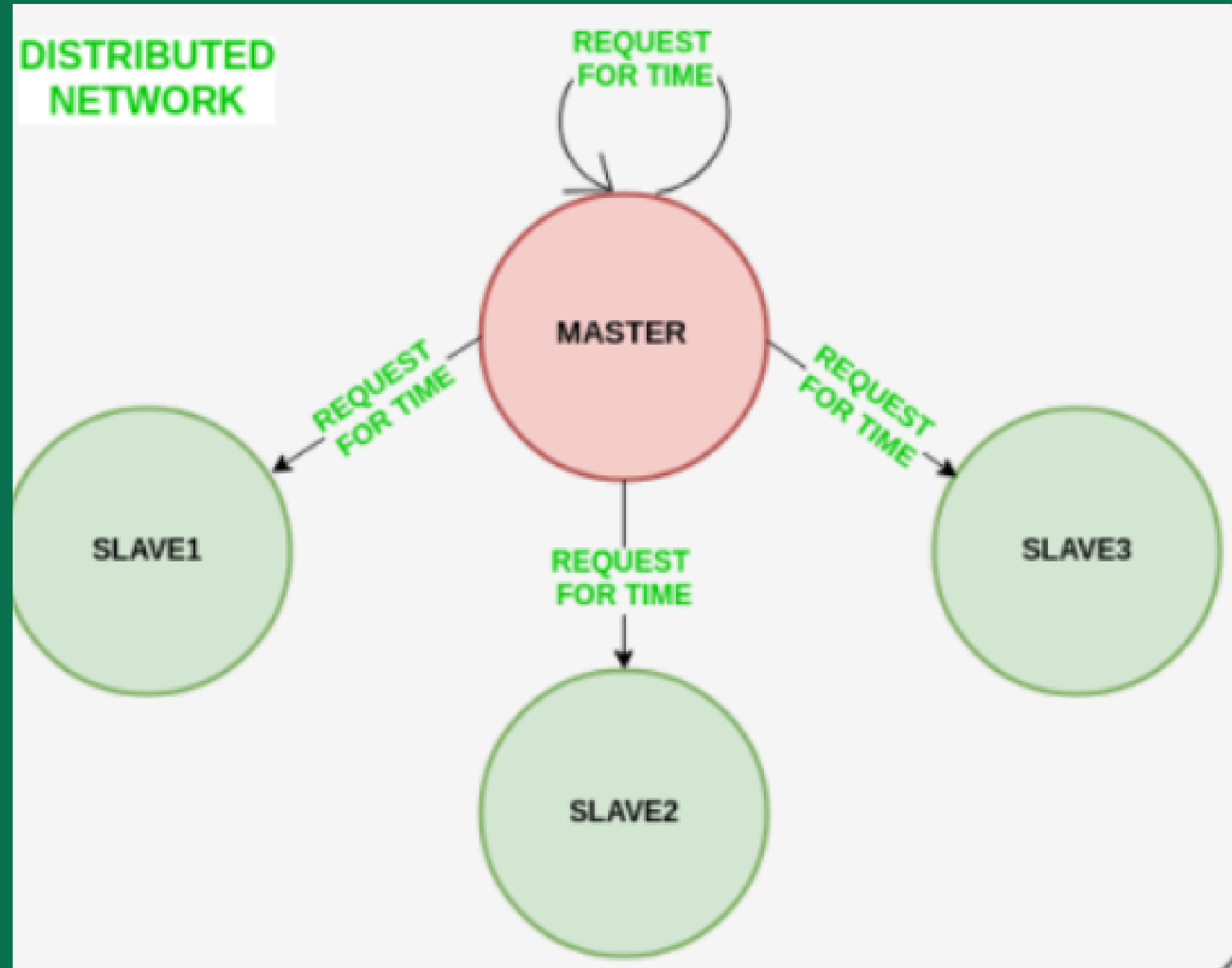
- This node is the main node in the network which acts as a master and the rest of the nodes act as slaves. The master node is chosen using an election process/leader election algorithm.

Master node periodically pings slaves nodes and fetches clock time at them using Cristian's algorithm.

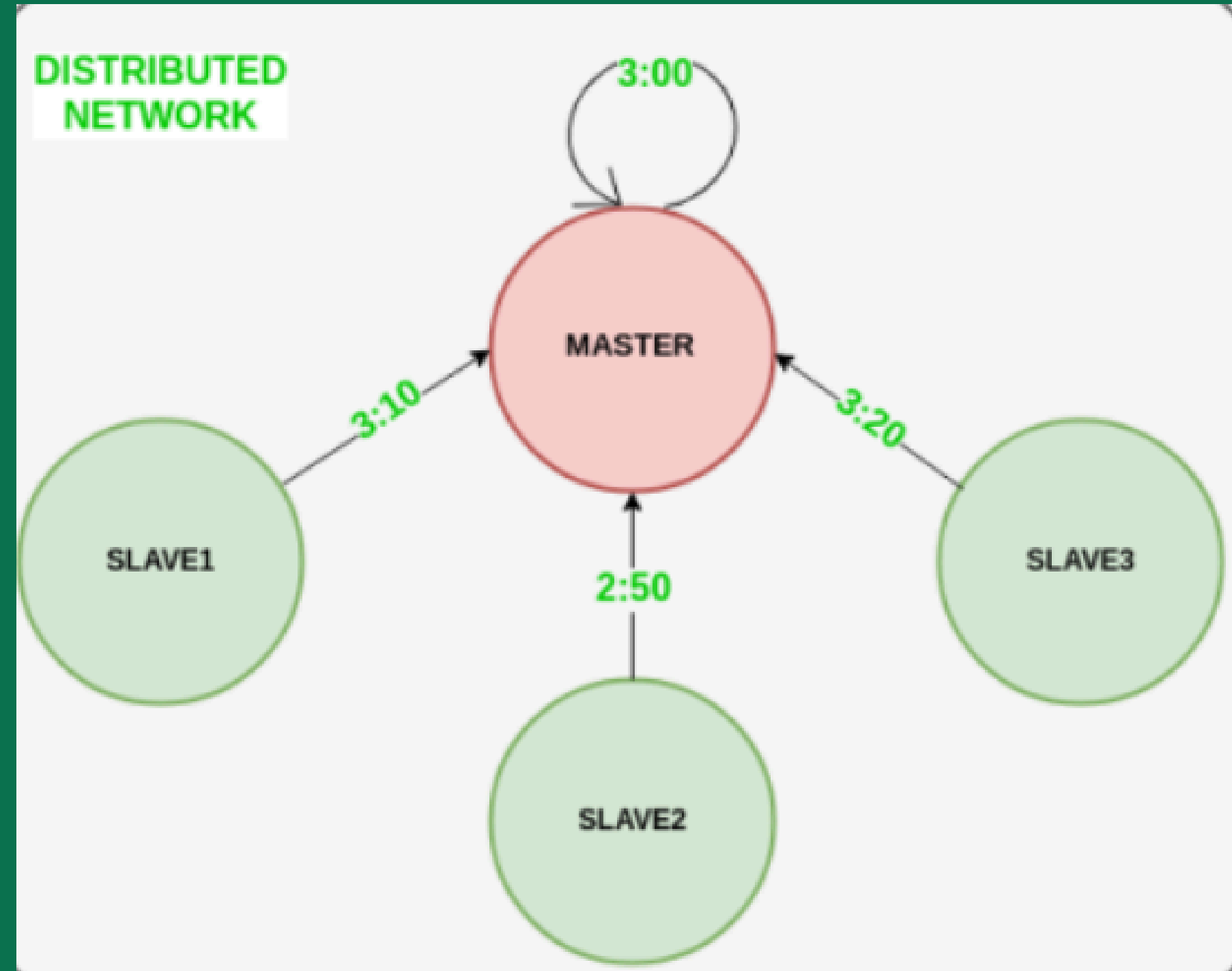
Master node calculates the average time difference between all the clock times received and the clock time given by the master's system clock itself.

- This average time difference is added to the current time at the master's system clock and broadcasted over the network

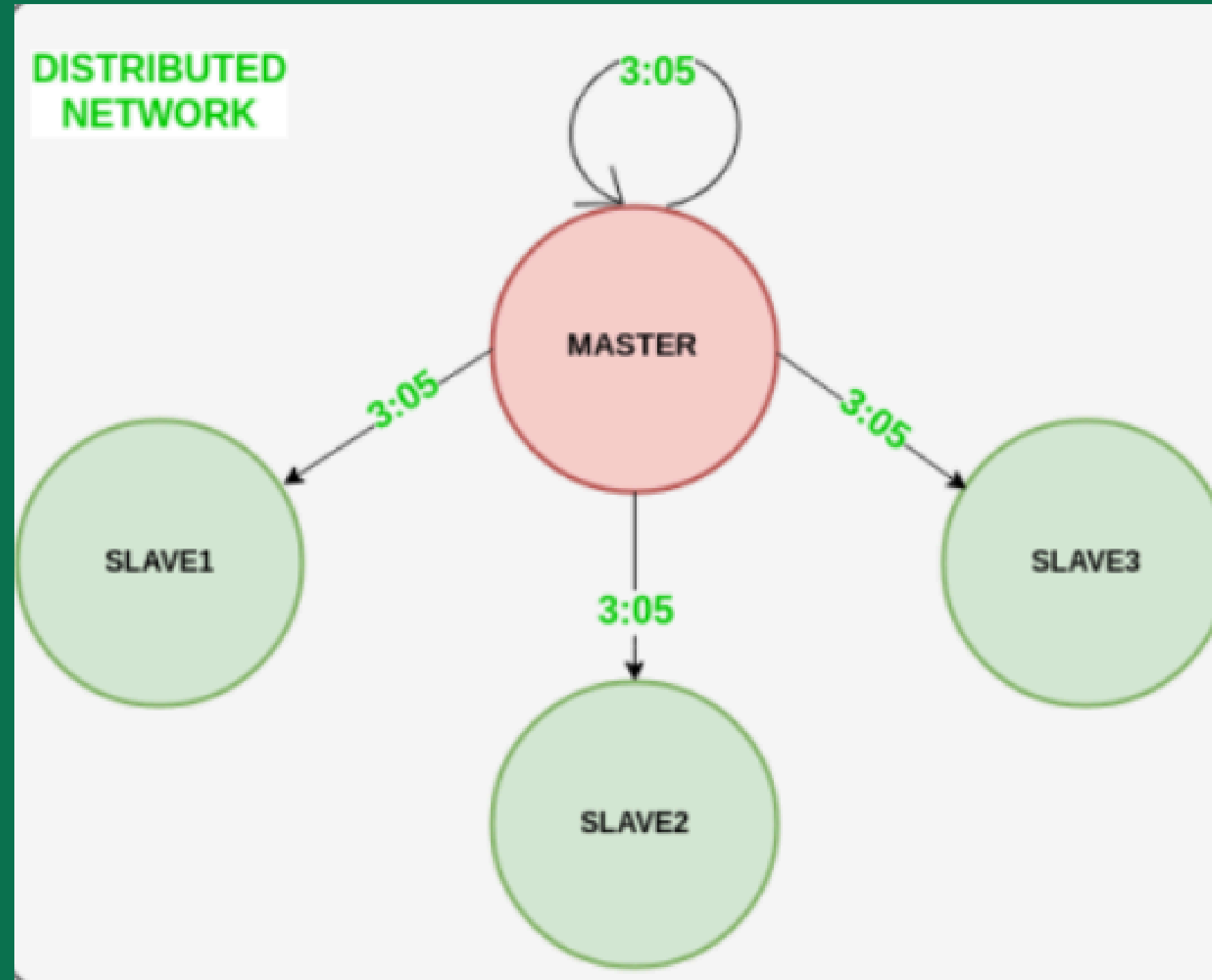
EXAMPLE



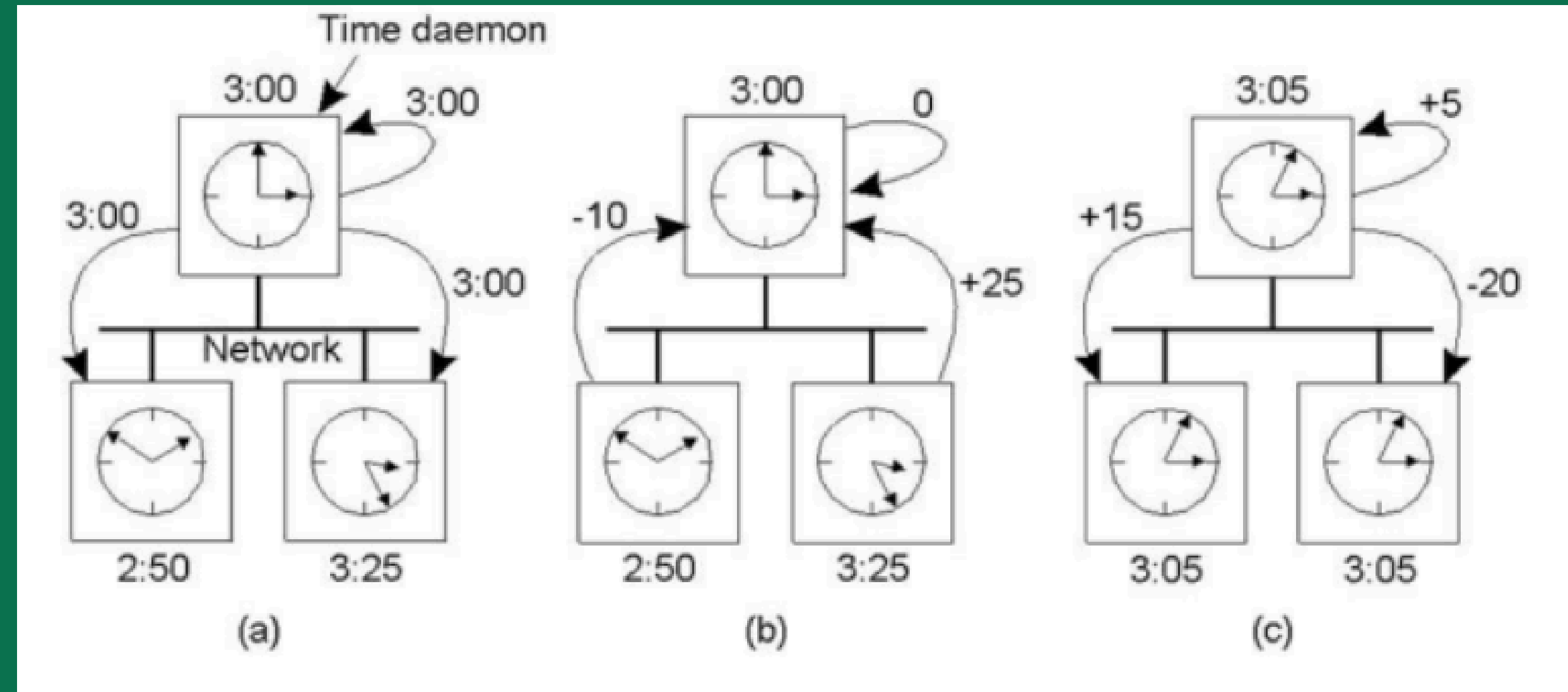
EXAMPLE



EXAMPLE



EXAMPLE



a) The time daemon asks all the other machines for their clock values

b) The machines answer

c) The time daemon tells everyone how to adjust their clock

Algorithm	Advantages	Disadvantages
Lamport Clocks	• Simple and easy to implement • Low overhead • Can be used in any distributed system	• Not suitable for real-time systems • Can cause unnecessary synchronization in some cases • May not preserve causality
Vector Clocks	• Better resolution of conflicts • More precise ordering of events • Efficient resolution of conflicts • Can detect concurrent events • Can be used in any distributed system	• Complexity increases with the number of nodes • May require additional storage and communication overhead • Vector size grows with the number of processes
Scalar Clocks	• Simple and efficient • No synchronization needed	• Not suitable for real-time systems • Limited resolution of conflicts • May not preserve causality
Berkeley's	• It is simple and easy to implement. • It can achieve high accuracy in clock synchronization. • Can be used to synchronize clocks in a distributed system	• It is dependent on the accuracy of the time server. • It assumes that the communication delay is known and constant, which may not be true in a real-world distributed system.



THANK YOU